

Projet Trolls et Châteaux : stratégie prudente – MIF26

*Intervenant : Fares FERCHICHE P2208151, Irfat FEJZULLAHU P2200031
et Max M'BEY P2511097*

Introduction

Dans ce projet, nous avons travaillé sur le jeu Trolls et Châteaux, un jeu à deux joueurs où chacun cherche à éloigner le troll de son propre château. Le jeu se déroule sur un chemin avec un château à chaque extrémité. À chaque tour, les deux joueurs choisissent secrètement un nombre de pierres à lancer. Celui qui lance le plus de pierres fait avancer le troll vers l'adversaire, tandis qu'en cas d'égalité, le troll reste sur place.

L'objectif du projet était de mettre en place une stratégie optimale qu'on appelle stratégie prudente. L'idée est de trouver une manière de jouer qui garantit le meilleur résultat possible même si l'adversaire joue intelligemment. Pour cela, le programme « `strategie_optimale.py` » modélise le jeu, calcule les valeurs des sous-jeux, utilise la programmation linéaire pour résoudre les situations et teste cette stratégie contre plusieurs stratégies plus simples.

Modélisation

Pour représenter le jeu, on définit un état avec la position du troll et les réserves de pierres des deux joueurs. Dans le code, cet état est représenté par « `Etat_Jeu` », tandis que les paramètres généraux du jeu sont regroupés dans « `Configuration_Jeu` ». Cette séparation rend le programme plus clair et permet de tester facilement plusieurs configurations.

Le gain est toujours calculé du point de vue du joueur de gauche : 1 pour une victoire, -1 pour une défaite et 0 pour un match nul. Cette convention permet de représenter le jeu comme un jeu à somme nulle.

Règles de transition

La fonction « `appliquer_tour` » applique les règles d'un tour. Elle compare les coups des deux joueurs, déplace le troll si nécessaire, puis retire les pierres utilisées.

La partie s'arrête si le troll atteint un château ou si un joueur n'a plus de pierres. Dans ce cas, le résultat est déterminé directement ou à l'aide de la règle finale, qui utilise les pierres restantes pour déplacer virtuellement le troll. Si le troll finit exactement au centre, le résultat est nul.

Le programme vérifie aussi les coups joués. Un coup invalide entraîne une défaite, sauf si les deux joueurs jouent un coup invalide.

Calcul de la stratégie prudente

Pour chaque état non terminal, le programme construit une matrice de gain. Les lignes correspondent aux coups possibles du joueur de gauche, et les colonnes aux coups possibles du joueur de droite. Chaque case contient soit le gain final, soit la valeur prudente du sous-jeu atteint.

La fonction « valeur_et_strategie » calcule récursivement la valeur d'un état. Elle traite d'abord les cas terminaux, puis construit la matrice des coups possibles et appelle « resoudre_jeu_matriciel ».

Le décorateur « lru_cache » permet d'éviter de recalculer plusieurs fois le même état. Cela rend le programme plus efficace, surtout lorsque les configurations deviennent plus grandes.

Programme linéaire

À chaque état, le jeu matriciel est résolu avec PuLP. Le joueur de gauche cherche une distribution de probabilité sur ses coups afin de maximiser son gain garanti, peu importe la réponse du joueur adverse.

Cette résolution donne donc la valeur prudente de l'état ainsi que la stratégie optimale à jouer. Pour le joueur de droite, le programme utilise la symétrie du jeu en inversant la position du troll et les réserves des deux joueurs.

Expérimentations

Les expériences suivantes ont été lancées avec la graine aléatoire 0. La stratégie prudente est jouée par le joueur de gauche.

Pour chaque configuration, la valeur prudente initiale est 0. Cela signifie que le joueur de gauche peut garantir au minimum un match nul. La distribution du premier coup indique ensuite les probabilités des coups optimaux.

- Configuration (3 cases, 3 pierres) :

Pour cette configuration, la stratégie optimale joue toujours 3 pierres au premier coup :

Coup (pierres)	Probabilité
3	1.0000

Affrontement	Victoires gauche	Victoires droite	Nuls	Score moyen gauche
Prudente vs aléatoire	691	0	309	0.691
Prudente vs agressive	1000	0	0	1.000
Prudente vs prudente	0	0	1000	0.000

Après simulation, on remarque que la stratégie prudente fait uniquement match nul contre elle-même. Contre les stratégies aléatoire et agressive, elle obtient de très bons résultats.

- Configuration (7 cases, 15 pierres) :

Coup (pierres)	Probabilité
2	0.4612
3	0.0488
4	0.4899

Résultats sur 1000 parties :

Affrontement	Victoires gauche	Victoires droite	Nuls	Score moyen gauche
Prudente vs aléatoire	956	34	10	0.922
Prudente vs agressive	1000	0	0	1.000
Prudente vs prudente	337	318	345	0.019

Les résultats montrent que la stratégie prudente gagne très souvent contre les stratégies simples. Contre elle-même, le score moyen reste proche de zéro, ce qui est cohérent avec la symétrie du jeu.

- Configuration (7 cases, 30 pierres) :

Coup (pierres)	Probabilité
1	0.1225
2	0.0542
3	0.1349
4	0.0663
5	0.1468
6	0.0766
7	0.1571

8	0.0825
9	0.1591

Résultats sur 200 parties :

Affrontement	Victoires gauche	Victoires droite	Nuls	Score moyen gauche
Prudente vs aléatoire	191	9	0	0.910
Prudente vs agressive	188	12	0	0.80
Prudente vs prudente	87	97	16	-0.050

Avec plus de pierres, la stratégie prudente devient plus variée. Elle répartit ses choix sur plusieurs coups, ce qui la rend moins prévisible pour l'adversaire.

Analyse

Le résultat le plus important est que les configurations symétriques ont une valeur prudente initiale nulle. Cela est logique, car les deux joueurs commencent avec les mêmes ressources et le troll est placé au centre. Aucun joueur n'a donc d'avantage au départ.

Cependant, la stratégie prudente reste très forte contre des stratégies moins efficaces. Elle ne se contente pas de viser le match nul : si l'adversaire joue mal, elle peut profiter de ses erreurs et gagner très souvent.

Les résultats contre une autre stratégie prudente sont proches de zéro. Cela montre que les distributions calculées jouent bien leur rôle : si les deux joueurs jouent prudemment, aucun ne peut vraiment prendre l'avantage.

On remarque aussi que lorsque le nombre de pierres augmente, la stratégie mélange davantage ses coups. Cela permet d'éviter d'être trop prévisible.

Limites et améliorations possibles

La principale limite du programme est le temps de calcul. Plus le nombre de pierres augmente, plus il y a d'états à explorer et de programmes linéaires à résoudre.

Une première amélioration serait de sauvegarder les valeurs déjà calculées, afin de ne pas recommencer tous les calculs à chaque lancement.

Une autre amélioration serait d'utiliser encore plus la symétrie du jeu. Certains états sont équivalents, ce qui pourrait permettre de réduire le nombre de calculs.

Enfin, pour le concours, il serait intéressant d'ajouter une partie adaptative. Le programme pourrait reconnaître si l'adversaire joue de façon aléatoire, agressive ou économe, puis adapter sa stratégie tout en gardant la stratégie prudente comme base sûre.

Conclusion

Ce projet nous a permis de modéliser le jeu Trolls et Châteaux et d'implémenter une stratégie prudente. Le programme calcule les valeurs des sous-jeux, utilise la programmation linéaire pour obtenir des stratégies mixtes, puis compare cette stratégie à plusieurs comportements simples.

Les expériences confirment les résultats attendus. Dans les configurations symétriques, la valeur prudente est nulle. Contre une autre stratégie prudente, le score moyen reste proche de zéro. Contre des stratégies moins rigoureuses, la stratégie prudente gagne très souvent.

Le programme répond donc à l'objectif du projet. L'amélioration principale à envisager concerne surtout les performances pour les grandes configurations.